

Runbook — Fase 2: Validação Técnica da Tool

Processo operacional para validação manual e futura automação por agente de IA

Versão: 1.0 | Data: 27/09/2026

Fluxo: Collection + contrato → Tool direta → MCP Client → Segurança → Resiliência → Performance → Observabilidade → Gate → Fase 3

1. Objetivo e Escopo

Objetivo: Certificar tecnicamente uma tool antes de sua avaliação pelo agente. A Fase 2 verifica se a tool é previsível, segura, resiliente, observável e tecnicamente aderente ao contrato.

Escopo: A validação é executada sem o agente. A Seção 1 chama a tool diretamente. A Seção 2 usa um MCP Client. As Seções 3 a 6 avaliam segurança, resiliência, performance e observabilidade. A avaliação de seleção da tool pelo agente fica para a Fase 4.

Princípio: Todo teste deve possuir: cenário, pré-condições, entrada, procedimento, resultado esperado, critério de aceite, resultado obtido, evidência e status.

2. Pré-requisitos

2.1 Artefatos obrigatórios: Disponibilizar documentação do schema de input/output; exemplos válidos e inválidos; Collection de Validação da Fase 1; resultados esperados por chamada; método de autenticação; timeouts; SLAs/SLOs; acesso ao ambiente de teste; owner técnico; link da documentação; link do dataset de treinamento; dashboard principal do Datadog.

2.2 Collection de Validação: A Collection deve conter chamadas reproduzíveis para o servidor/tool, parâmetros necessários e resultado esperado de cada cenário. Pode ser Postman, Insomnia ou formato equivalente compartilhado. Deve possuir versão, responsável e data de atualização.

2.3 Critério de entrada: Não iniciar a execução quando faltar um artefato obrigatório, quando os resultados esperados forem ambíguos ou quando o ambiente de teste não estiver acessível. Registrar o bloqueio.

3. Seção 1 — Validação do Contrato Técnico

Objetivo: Comprovar, em chamada direta à tool, que inputs, outputs e erros seguem o contrato definido, sem envolver MCP Client ou agente.

3.1 Campos obrigatórios: Validar uma chamada válida e repetir removendo cada campo obrigatório individualmente. Verificar rejeição controlada e aderência ao erro esperado.

Aceite: Todos os campos obrigatórios funcionam quando presentes; chamadas sem campos obrigatórios são rejeitadas conforme contrato; nenhum campo obrigatório é ignorado silenciosamente.

3.2 Tipos de dados: Executar valores válidos e alterar individualmente tipos de string, número, boolean, array e objeto, conforme aplicável.

Aceite: Tipos válidos são aceitos; tipos incompatíveis são rejeitados ou tratados exatamente como definido no contrato.

3.3 Restrições e valores: Testar limites, enums, formatos, strings vazias, nulos, valores fora do intervalo e combinações incompatíveis.

Aceite: Restrições documentadas são efetivamente aplicadas e o comportamento para valores inválidos é determinístico.

3.4 Output: Comparar campos, tipos e estrutura do response real com o schema e com os resultados esperados da Collection.

Aceite: Output aderente ao schema; campos obrigatórios presentes; tipos e estrutura corretos.

3.5 Ausência de dados: Executar cenário sem dados e validar status, estrutura, mensagem e campos opcionais.

Aceite: Ausência de dados não produz resposta ambígua nem quebra o contrato.

3.6 Erros: Executar cenários inválidos e verificar formato, consistência e segurança das mensagens de erro.

Aceite: Erro previsível, estruturado quando aplicável, sem exposição de secrets ou dados sensíveis.

Evidências: Request, response, comparação com esperado, schema validado, logs/traces quando necessários e identificação do caso da Collection.

4. Seção 2 — Validação da Integração MCP

Objetivo: Comprovar que o comportamento validado diretamente na Seção 1 é preservado quando a tool é acessada pelo protocolo MCP.

4.1 Descoberta: Conectar com MCP Client e listar as tools. Comparar nome, descrição, schema e parâmetros publicados com a definição aprovada.

Aceite: Tool é descoberta; identidade e schema publicados são corretos e suficientes para execução.

4.2 Chamada via MCP: Executar os mesmos cenários relevantes da Collection usando MCP Client e comparar o resultado com a chamada direta.

Aceite: A chamada é encaminhada à tool correta e o resultado é equivalente ao comportamento esperado da Seção 1.

4.3 Argumentos: Testar strings, números, booleanos, arrays, objetos, opcionais e nulos conforme aplicável.

Aceite: Nenhum argumento é perdido, convertido indevidamente ou alterado de forma incompatível durante o transporte MCP.

4.4 Erros MCP: Executar chamadas inválidas e cenários de falha para observar como o MCP Client recebe e interpreta erros.

Aceite: Erros são propagados de forma controlada, identificável e sem exposição de informação sensível.

4.5 Consistência: Comparar: chamada direta → resultado e MCP Client → resultado.

Aceite: Diferenças não justificadas pelo protocolo ou pelo contrato são falhas.

5. Seção 3 — Validação de Segurança

Objetivo: Verificar autenticação, autorização, proteção de dados e políticas de segurança.

5.1 Autenticação: Testar credencial válida, inválida, ausente e expirada quando aplicável.

Aceite: Apenas credenciais válidas e vigentes obtêm acesso conforme a política.

5.2 Autorização: Executar com identidade autorizada e não autorizada, incluindo tentativa de acesso a recurso não permitido.

Aceite: A autorização é aplicada no ponto correto e acesso indevido é bloqueado.

5.3 Dados sensíveis: Inspeccionar responses, erros, logs e traces em cenários normais e de falha.

Aceite: Não há exposição indevida de secrets, tokens, credenciais ou dados sensíveis.

5.4 Secrets: Verificar configuração, logs e respostas para garantir que secrets não estejam hardcoded ou retornados.

Aceite: Nenhum secret aparece em código/configuração indevida, response, log ou trace.

5.5 OPA: Quando houver policy OPA aplicável, executar a avaliação da policy contra a API/tool e registrar input, decisão e evidência.

Aceite: A API atende às policies obrigatórias; violações são identificadas e impedem aprovação quando classificadas como bloqueantes.

6. Seção 4 — Resiliência e Tratamento de Erros

Objetivo: Verificar comportamento controlado diante de falhas, indisponibilidade e respostas inesperadas.

6.1 Timeout: Provocar ou simular dependência lenta e verificar timeout, tempo até falha e tipo de erro.

Aceite: Não há espera indefinida; timeout respeita configuração ou SLO definido.

6.2 Dependência indisponível: Simular API, banco ou serviço downstream indisponível.

Aceite: Falha é tratada de forma previsível, sem crash ou comportamento indefinido.

6.3 Resposta inválida: Simular JSON inválido, campo ausente, schema inesperado e tipo inesperado.

Aceite: Tool rejeita ou trata resposta inválida de forma controlada e observável.

6.4 Retry: Verificar existência, quantidade, backoff e condições de retry.

Aceite: Retry ocorre somente quando apropriado, respeita limites e não amplifica falhas ou efeitos colaterais.

6.5 Idempotência: Para operações mutáveis, repetir a mesma chamada ou simular retry.

Aceite: Retries não geram duplicidade ou efeitos colaterais indevidos quando a operação exige idempotência.

7. Seção 5 — Performance e Capacidade

Objetivo: Medir o comportamento quantitativo da tool e compará-lo com os SLAs/SLOs definidos na Fase 1.

7.1 Latência: Executar conjunto representativo de chamadas e calcular média, mediana, P95, P99 e máximo.

Aceite: Métricas ficam dentro dos limites acordados; desvios devem ser classificados e justificados.

7.2 Taxa de erro: Executar volume definido de chamadas e calcular erros/total.

Aceite: Taxa de erro atende ao limite estabelecido para o ambiente.

7.3 Concorrência: Executar chamadas simultâneas em níveis progressivos.

Aceite: Tool mantém comportamento correto e métricas dentro dos limites sob concorrência esperada.

7.4 Carga: Aumentar gradualmente o volume até o limite esperado ou até identificar degradação.

Aceite: Capacidade observada atende à demanda de referência ou o limite é documentado.

7.5 Degradação: Registrar o ponto em que latência/erro/throughput começam a degradar.

Aceite: Ponto de degradação conhecido e compatível com a capacidade esperada; ausência de comportamento catastrófico.

8. Seção 6 — Observabilidade

Objetivo: Comprovar que uma falha pode ser detectada, correlacionada e diagnosticada.

8.1 Logs: Executar chamadas de sucesso e falha e localizar registros correspondentes.

Aceite: Logs possuem contexto suficiente, como timestamp, operação, correlation/request ID e resultado, sem dados sensíveis.

8.2 Métricas: Verificar chamadas, erros, latência, throughput, timeout e retry quando aplicável.

Aceite: Métricas existem, são atualizadas e permitem identificar degradação.

8.3 Tracing: Executar chamada e acompanhar o fluxo MCP Server → Tool → dependência.

Aceite: Trace permite localizar o componente responsável por uma falha.

8.4 Datadog: Abrir o dashboard principal informado na Fase 1 e executar cenários conhecidos para localizar os sinais correspondentes.

Aceite: Dashboard apresenta sinais relevantes e permite correlacionar uma execução com logs/métricas/traces.

9. Evidências e Registro

Registro mínimo: Cada teste deve registrar ID, seção, cenário, pré-condições, input, procedimento, resultado esperado, resultado obtido, status, evidência, timestamp, ambiente e observações.

Status: PASS: todos os critérios do teste atendidos. FAIL: pelo menos um critério não atendido. BLOCKED: execução não foi possível por pré-condição/infraestrutura. NOT APPLICABLE: item não se aplica, com justificativa.

Regra: Não marcar PASS por ausência de evidência. Um teste executado sem evidência verificável deve permanecer como FAIL ou BLOCKED, conforme a causa.

10. Gate de Aprovação da Fase 2

Critério geral: A Fase 2 é aprovada quando todas as seções aplicáveis possuem critérios de aceite atendidos, evidências registradas e nenhuma falha crítica ou bloqueante permanece aberta.

FAIL: Qualquer falha crítica de contrato, segurança, integração MCP, resiliência ou outro requisito obrigatório impede o avanço até correção ou exceção formal.

BLOCKED: Bloqueios devem ser resolvidos antes da aprovação, salvo exceção formal definida pelo processo.

Saída: APROVADO PARA FASE 3 ou RETORNAR PARA CORREÇÃO. O resultado deve incluir resumo das falhas, evidências e pendências.

11. Modelo de Caso de Teste

Template: ID: F2-Sx-Tyy | Cenário: ... | Pré-condições: ... | Input: ... | Procedimento: ... | Resultado esperado: ... | Critério de aceite: ... | Resultado obtido: ... | Evidência: ... | Status: PASS/FAIL/BLOCKED/N/A.

Exemplo: Cenário: chamada sem customer_id. Input: payload sem customer_id. Resultado esperado: rejeição conforme contrato. Critério de aceite: chamada rejeitada e erro correspondente ao contrato. Evidência: request/response + log. Status: PASS se todos os critérios forem atendidos.

12. Automação Futura

Princípio: O Runbook deve ser executável manualmente hoje e estruturado para automação amanhã.

Automatizável: Validação de schema, execução da Collection, comparação de outputs, descoberta MCP, testes de argumentos, chamadas negativas, coleta de métricas, consulta de observabilidade e consolidação de evidências.

Agente de IA: Um futuro agente pode receber os artefatos da Fase 1, executar os testes permitidos, interpretar resultados contra critérios explícitos e gerar o relatório. A aprovação final deve permanecer sujeita às regras de governança definidas pela equipe.

Checklist Executivo

Seção	Objetivo	Critério de aceite	Status
1	Contrato técnico	Input/output/erros aderentes ao contrato	
2	Integração MCP	Descoberta, chamadas e erros MCP corretos	
3	Segurança	Acesso e proteção de dados conforme políticas	
4	Resiliência	Falhas tratadas sem comportamento indefinido	
5	Performance	SLA/SLO e capacidade atendidos	
6	Observabilidade	Logs, métricas, traces e Datadog utilizáveis	

Resultado final: ■ APROVADO PARA FASE 3 ■ RETORNAR PARA CORREÇÃO